Phát hiện phương tiện vượt đèn đỏ

Nguyễn Tuấn Anh
Lớp 124231
Bài tập lớn môn Deep Learning

Abstract

Bài báo trình bày hệ thống thị giác máy tính phát hiện phương tiện vượt đèn đỏ từ video camera giao thông cố định. Hệ thống kết hợp nhận diện phương tiện bằng YOLO, gán định danh theo thời gian bằng ByteTrack, phân loại trạng thái đèn và kiểm tra quỹ đạo cắt vạch dừng theo hướng di chuyển. Một sự kiện chỉ được xác nhận khi xe nằm trước vạch, cắt đoạn vạch hữu hạn trong trạng thái đèn đỏ và tiếp tục xuất hiện phía sau vạch đủ số khung hình quy định. Mỗi sự kiện được lưu cùng thời điểm, ID, ảnh toàn cảnh và ảnh cắt phương tiện. Detector được đánh giá trên tập con COCO gồm 1.000 ảnh train, 200 ảnh validation và 200 ảnh test với năm lớp giao thông. Trong cùng giao thức, YOLOv8s đạt mAP@0.50:0.95 là 0,318 và F1 là 0,529, cao nhất trong ba cấu hình SSD300-VGG16, YOLOv8s và YOLO11x; SSD300-VGG16 đạt tốc độ cao nhất, 166,8 FPS trên NVIDIA H100. Kết quả cho thấy YOLOv8s cân bằng tốt nhất giữa độ chính xác, tốc độ và độ phức tạp trên tập thử. Bài báo đồng thời phân tích giới hạn chuyển miền, sai số tích lũy và yêu cầu đánh giá ở mức sự kiện trước khi triển khai thực tế.

1 Giới thiệu

Vượt đèn đỏ có nguy cơ tạo xung đột trực tiếp tại nút giao. Giám sát thủ công nhiều luồng camera tốn nhân lực, khó bảo đảm tính nhất quán và thường chỉ cho phép xem lại sau sự cố. Một hệ thống tự động khoanh vùng sự kiện nghi vấn, giữ bằng chứng hình ảnh và xuất dữ liệu có cấu trúc sẽ hỗ trợ đáng kể cho người vận hành.

Bài toán không dừng ở nhận diện xe trong một ảnh. Hệ thống phải đồng thời xác định phương tiện nào đang được theo dõi, đèn nào điều khiển đúng làn, xe có thực sự đi qua vạch theo chiều bị giám sát hay không, và bằng chứng nào đủ để kiểm tra lại quyết định. Sai số của bất kỳ mô-đun nào cũng có thể lan truyền đến kết quả cuối. Mất ID có thể làm bỏ sót sự kiện; giữ trạng thái đỏ sau chuyển pha có thể tạo báo sai.

Đề tài xây dựng pipeline xử lý video hoàn chỉnh cho camera cố định. Các đóng góp chính gồm: (i) cấu hình hình học riêng cho từng video/camera bằng dấu vân tay nội dung; (ii) phát hiện cắt vạch có hướng trên đoạn thẳng hữu hạn; (iii) cơ chế xác nhận nhiều khung hình và xử lý trạng thái đèn không chắc chắn theo hướng thận trọng; (iv) báo cáo sự kiện có khả năng truy vết; và (v) thực nghiệm thống nhất so sánh ba detector trên cùng dữ liệu và phần cứng.

2 Cơ sở lý thuyết và công trình liên quan

2.1 Nhận diện đối tượng

YOLO định nghĩa nhận diện đối tượng như bài toán hồi quy hợp nhất từ ảnh đến hộp giới hạn và xác suất lớp, tạo nền tảng cho các detector một giai đoạn tốc độ cao [1]. SSD cũng loại bỏ bước đề xuất vùng, sử dụng các hộp mặc định ở nhiều tỉ lệ và nhiều tầng đặc trưng để dự đoán lớp cùng độ lệch vị trí [2]. Hai hướng tiếp cận phù hợp với video giao thông, nơi thông lượng quan trọng bên cạnh độ chính xác.

Đề tài dùng SSD300-VGG16 làm baseline; YOLOv8s và YOLO11x đại diện cho hai cấu hình YOLO có quy mô khác nhau. Tập thử được trích từ COCO [3] và giữ năm lớp *bicycle*, *car*, *motorcycle*, *bus*, *truck*.

2.2 Theo dõi đa đối tượng

Phát hiện độc lập từng frame không tạo định danh bền vững. ByteTrack liên kết quỹ đạo bằng cả các hộp có độ tin cậy thấp thay vì loại bỏ chúng ngay [4]. Điều này hữu ích khi xe bị che khuất hoặc ở xa camera. Hệ thống dùng ByteTrack để nhận ID, rồi áp dụng logic cắt vạch trên chuỗi điểm neo của từng ID.

2.3 Nhận diện trạng thái đèn

Nhận diện đèn thường kết hợp phát hiện vùng đèn và phân loại màu. Kích thước nhỏ, chói sáng, ngược sáng và khác biệt thiết kế khiến chuyển miền trở thành thách thức [6]. Hệ thống dùng ROI đèn gắn với từng làn và MobileNetV2 của Autoware ở định dạng ONNX. MobileNetV2 sử dụng inverted residual và linear bottleneck để giảm chi phí tính toán [5]. Khi không đủ tự tin, trạng thái được đưa về UNKNOWN, không suy diễn thành đỏ.

3 Phát biểu bài toán

Đầu vào là chuỗi frame I_t. Detector sinh tập

$$D_t = \{(b_i, c_i, p_i)\}_{i=1}^{n_t}, \tag{1}$$

với b_i là hộp, c_i là lớp và p_i là độ tin cậy. Tracker ánh xạ mỗi quan sát sang ID k và tạo quỹ đạo $\mathcal{T}_k = \{q_k^t\}$; q_k^t là điểm neo giữa cạnh đáy hộp xe.

Mỗi làn được biểu diễn bởi cổng $G = (L, \boldsymbol{d}, R, m)$ gồm đoạn vạch $L = (A, B)$, hướng hợp lệ $\boldsymbol{d}$, vùng đèn R và hướng chuyển động m. Trạng thái đèn sau lọc thời gian là $s_t \in \{\text{RED}, \text{YELLOW}, \text{GREEN}, \text{UNKNOWN}\}$. Với khoảng cách có dấu

$$\delta(q, L) = \frac{(B - A) \times (q - A)}{\|B - A\|_2}, \tag{2}$$

trong đó $\times$ là tích có hướng hai chiều. ứng viên xuất hiện khi dấu của δ đổi theo chiều quy định và đoạn nối hai quan sát liên tiếp giao với chính đoạn AB. Sự kiện chỉ được xác nhận nếu $s_t = \text{RED}$ tại lúc cắt, xe tiếp tục ở phía sau vạch tối thiểu K frame và khoảng cách lớn hơn biên ϵ.

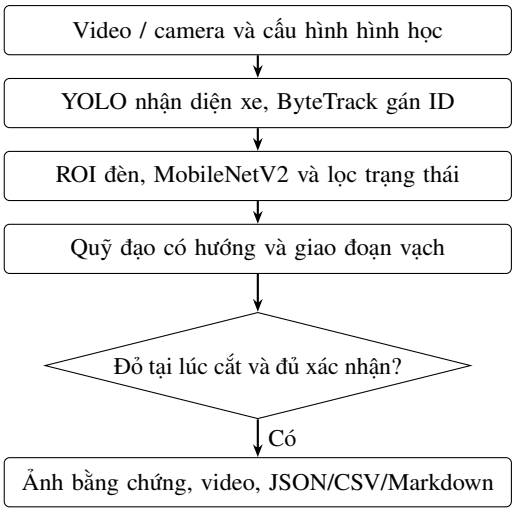
4 Kiến trúc hệ thống

4.1 Pipeline tổng thể

Hình 1 mô tả luồng xử lý. Camera được hiệu chỉnh một lần để xác định vạch, hướng đi và ROI đèn. Trong mỗi frame, hệ thống nhận diện phương tiện, cập nhật ID, phân loại đèn và hợp nhất kết quả tại bộ máy luật. Đầu ra gồm video, ảnh bằng chứng và báo cáo JSON/CSV/Markdown.

4.2 Hiệu chỉnh theo camera

Cấu hình gồm hai đầu mút vạch, hộp đèn điều khiển và điểm chỉ hướng phía trước vạch. Nó được gắn với SHA-256 của video, kích thước frame và ID camera, tránh dùng nhầm tọa độ giữa các góc quay. Cách biểu diễn không giả định vạch ngang theo trục ảnh.



Hình 1: Pipeline phát hiện sự kiện vượt đèn đỏ.

Hệ thống có mô-đun đề xuất vạch dựa trên Hough, hướng quỹ đạo, cụm xe dừng, độ dài và tương phản sơn, nhưng chỉ dùng thử nghiệm. Chế độ chính xác ưu tiên hình học do người dùng xác nhận vì vạch qua đường, mép đường và bóng sáng có thể giống vạch dừng.

4.3 Nhận diện và theo dõi xe

Detector giữ năm lớp phương tiện. Cấu hình video hiện tại dùng YOLO11x pretrained COCO, ảnh 960 và ngưỡng 0,10 để ByteTrack còn quan sát hộp yếu trong bước liên kết. Điểm neo mặc định $(0{,}5; 1{,}0)$ tương ứng giữa cạnh đáy hộp.

Mỗi ID chỉ được ghi một lần trong một lượt chạy. Track mất quá 0,20 giây bị loại khỏi trạng thái chờ. Nếu mất quan sát đúng lúc cắt vạch, cả quan sát trước và sau khoảng trống đều phải ở đèn đỏ. Điều kiện này hạn chế suy diễn qua đoạn video thiếu bằng chứng.

4.4 Trạng thái đèn

ROI đèn được đặt riêng theo làn để tránh chọn nhầm đèn rẽ hoặc đèn của hướng giao cắt. Crop được chuẩn hóa về 224×224 và phân loại bởi MobileNetV2 với ngưỡng 0,80. Trạng thái mới phải lặp ba frame trước khi chấp nhận. Trong chuyển tiếp hoặc khi các đèn mâu thuẫn, đầu ra là UNKNOWN. Hệ thống không giữ RED khi mất tín hiệu; lựa chọn này có thể giảm recall nhưng tránh báo sai do trạng thái cũ bị kéo dài.

4.5 Cắt vạch và xác nhận

Thuật toán lưu vị trí cuối ở trước vạch cho từng cặp (ID, cổng). Khi điểm mới ở sau vạch, nó kiểm tra giao thật giữa đoạn quỹ đạo và đoạn vạch hữu hạn.

Thành phần	Thiết lập chính
Xe	5 lớp; confidence 0,10; ảnh 960
Tracker	ByteTrack; mất tối đa 0,20 giây
Đèn	MobileNetV2 ONNX; confidence 0,80
Ổn định đèn	Xác nhận trạng thái mới 3 frame
Cắt vạch	Giao đoạn có hướng; biên 2 pixel
Vi phạm	Đỏ tại lúc cắt; xác nhận 3 frame
Báo cáo	MP4, JSON, CSV, Markdown, JPEG

Bảng 1: Cấu hình pipeline video.

Ứng viên đỏ phải duy trì phía sau vạch ba frame và vượt biên hai pixel. Nếu xe quay lại trước khi đủ xác nhận, ứng viên bị hủy.

Sự kiện lưu frame cắt, frame xác nhận, thời gian video, trạng thái đèn, ID cổng, ID xe, lớp, confidence và hai ảnh. Tách frame cắt khỏi frame xác nhận giữ đúng mốc thời gian thay vì gán thời điểm muộn do điều kiện nhiều frame.

5 Cài đặt

Hệ thống được hiện thực bằng Python, OpenCV, PyTorch/Ultralytics và ONNX Runtime. main.py điều phối pipeline; nhận diện đèn, cắt vạch, luật vi phạm, cấu hình cảnh và báo cáo được tách thành mô-đun. Mỗi lượt chạy có thư mục riêng, kèm cấu hình, hash checkpoint, số frame dự kiến, số frame đã xử lý và cờ hoàn tất.

Công cụ đánh giá sự kiện chỉ chấp nhận lượt chạy hoàn tất. Nó ghép một-một dự đoán và ground truth theo cổng với sai lệch thời gian tối đa 0,5 giây; dự đoán trùng được tính false positive.

6 Thiết kế thực nghiệm

6.1 Dữ liệu và huấn luyện

Tập con COCO gồm 1.000 ảnh train với 3.539 đối tượng, 200 ảnh validation với 685 đối tượng và 200 ảnh test với 762 đối tượng. Test không được dùng để chọn epoch hoặc siêu tham số. Ba mô hình dùng cùng thứ tự lớp và phép chia dữ liệu.

Mỗi mô hình được huấn luyện 80 epoch từ trọng số pretrained, seed 42. SSD300-VGG16 dùng ảnh 300×300, batch 8 và learning rate ban đầu 0,002. Hai YOLO dùng ảnh 640×640, batch 8 và cấu hình tối ưu mặc định. Checkpoint tốt nhất được chọn bằng mAP@0.50:0.95 trên validation.

6.2 Chỉ số

Độ chính xác hộp dùng COCO mAP@0.50:0.95 và mAP@0.50. Với confidence 0,25 và IoU 0,50:

$$P = \frac{TP}{TP + FP}, \qquad R = \frac{TP}{TP + FN}, \tag{3}$$

$$F_1 = \frac{2PR}{P + R}. \tag{4}$$

Tốc độ được đo batch 1, FP32 trên NVIDIA H100 sau warm-up và đồng bộ CUDA, gồm tiền xử lý model, suy luận và NMS nhưng không gồm đọc video, tracking hay ghi báo cáo. FPS vì vậy là tốc độ detector, không phải tốc độ đầu-cuối.

7 Kết quả và đánh giá

7.1 So sánh định lượng

Bảng 2 cho thấy YOLOv8s đạt cao nhất ở mAP, precision, recall và F1. So với SSD, mAP@0.50:0.95 tăng 0,061 điểm tuyệt đối và F1 tăng 0,084 điểm. YOLO11x có mAR100 cao nhất nhưng thấp hơn YOLOv8s ở mAP và F1.

SSD nhanh hơn YOLOv8s khoảng 39,1% nhưng giảm chất lượng hộp. YOLO11x có tham số gấp khoảng 5,1 lần YOLOv8s, trong khi mAP thấp hơn 0,009 và tốc độ chỉ bằng 54,3%. Quy mô lớn hơn không mang lại lợi ích tương ứng trên tập hiện tại.

YOLOv8s nằm gần biên tối ưu thực dụng: mAP cao nhất, 119,9 FPS và 11,1 triệu tham số. Tuy nhiên, pipeline video hiện dùng YOLO11x pretrained COCO vì kiểm tra trực quan ở camera mục tiêu cho thấy checkpoint tùy biến nhỏ có thể sinh hộp giả trên mặt đường trống. Quyết định triển khai phải dựa trên dữ liệu camera được gán nhãn, không chỉ tập con COCO.

7.2 Chi phí huấn luyện

YOLOv8s huấn luyện trong 9,2 phút, checkpoint 21,5 MB và peak VRAM 2.654 MB. SSD cần 16,4 phút, checkpoint 92,6 MB và 1.090 MB. YOLO11x cần 27,4 phút, checkpoint 109,1 MB và 7.348 MB. YOLOv8s đồng thời có checkpoint nhỏ nhất và thời gian huấn luyện ngắn nhất.

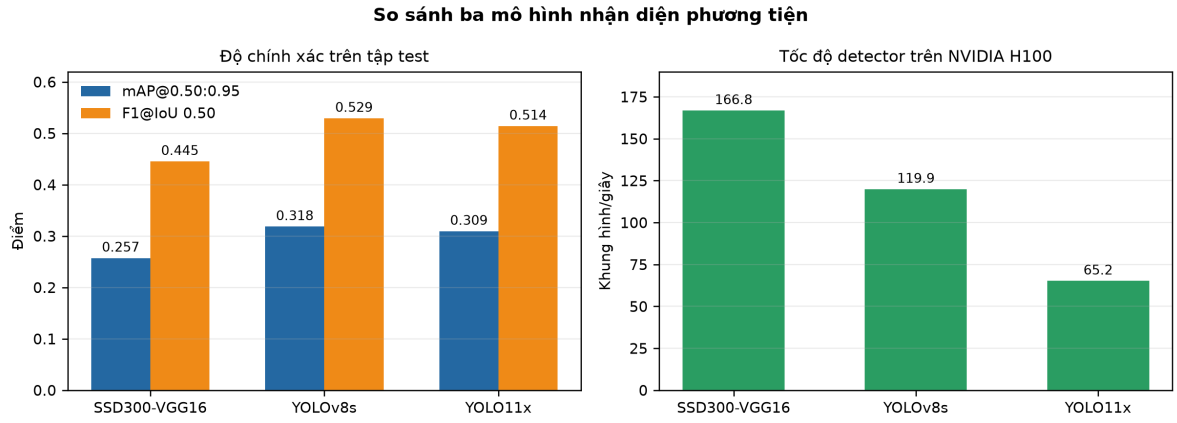
Đây là so sánh cấu hình thực dụng, không phải thí nghiệm cô lập kiến trúc: kích thước đầu vào, augmentation và tối ưu giữa SSD và YOLO khác nhau. Kết quả chỉ mô tả các cấu hình đã chạy.

7.3 Đánh giá định tính sự kiện

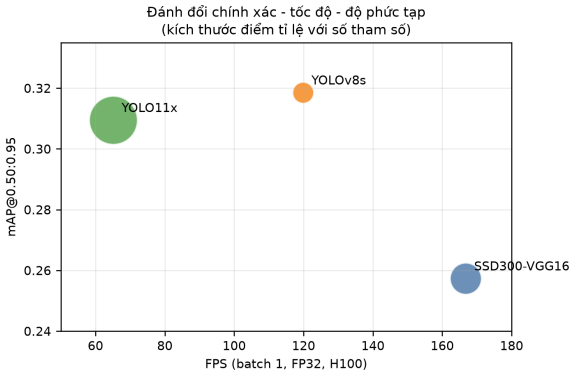
Hình 4 minh họa hệ thống duy trì ID, gắn trạng thái đỏ, kiểm tra xe tiến qua vạch và chỉ đổi nhãn sau

Mô hình	mAP50–95	mAP50	mAR100	Precision	Recall	F1	FPS	Tham số (M)
SSD300-VGG16	0,257	0,433	0,385	0,544	0,385	0,445	**166,8**	24,3
YOLOv8s	**0,318**	**0,519**	0,460	**0,570**	**0,497**	**0,529**	119,9	**11,1**
YOLO11x	0,309	0,513	**0,474**	0,554	0,484	0,514	65,2	56,9

Bảng 2: Kết quả trên 200 ảnh test; P/R/F1 dùng confidence 0,25 và IoU 0,50.



Hình 2: So sánh độ chính xác và tốc độ. YOLOv8s dẫn đầu về mAP và F1; SSD nhanh nhất.



Hình 3: Đánh đổi chính xác–tốc độ; kích thước điểm tỉ lệ số tham số.

đủ xác nhận. Việc hiển thị vạch, ROI làn, hộp, ID và đèn giúp phát hiện lỗi hình học hoặc liên kết.

Trong lượt chạy hoàn tất trên red1.mp4, hệ thống xử lý đủ 203/203 frame ở 30 FPS và tạo hai sự kiện confirmed_red_crossing. Sự kiện đầu cắt ở frame 56, xác nhận frame 58, confidence 0,77; sự kiện sau cắt ở frame 89, xác nhận frame 91, confidence 0,89. Độ trễ hai frame phù hợp điều kiện ba quan sát liên tiếp tính cả frame ứng viên.

Ảnh toàn cảnh quan trọng vì crop xe không chứng minh được trạng thái đèn, hướng đi hay quan hệ với vạch. Hai sự kiện này là đầu ra vận hành cần kiểm tra, chưa phải ground truth độc lập; không thể tự suy ra precision hoặc recall toàn hệ thống.

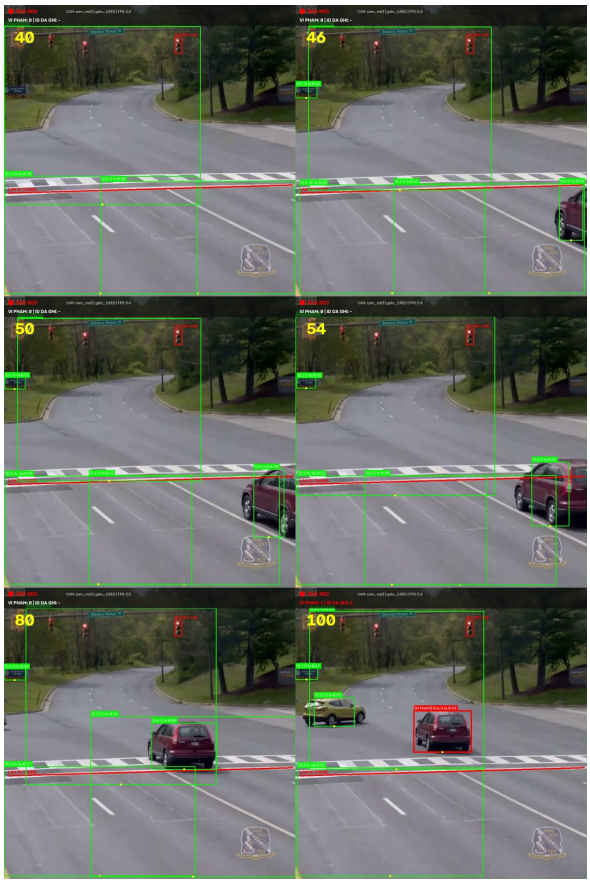
8 Phân tích sai số và hạn chế

Detector có thể bỏ sót xe nhỏ, xe che khuất hoặc nhầm bóng. Tracker có thể đổi ID sau che khuất dài. Classifier đèn chịu ảnh hưởng bởi chói sáng, mưa, đèn mũi tên và khác biệt khu vực. Vạch sai vài pixel có thể đổi frame cắt; chọn nhầm cổng hoặc hướng làm luật sự kiện mất ý nghĩa.

Các kiểm soát hiện tại giảm nhưng không loại bỏ sai số. UNKNOWN hạn chế báo sai do đèn mơ hồ; giao đoạn hữu hạn loại quỹ đạo cắt phần kéo dài ngoài làn; xác nhận nhiều frame loại dao động gần vạch; dấu vân tay video ngăn dùng nhầm cấu hình. Ảnh bằng chứng và log hỗ trợ kiểm tra phần còn lại.

Thí nghiệm detector chỉ dùng một seed và tập con COCO nhỏ nên chưa có khoảng tin cậy. COCO không đại diện đầy đủ cho mật độ xe máy, kiểu đèn, thời tiết và camera Việt Nam. Các chỉ số hộp chưa đo độ ổn định ID, màu đèn, sai số vạch hay chất lượng sự kiện. Mô hình Autoware chủ yếu học từ đèn Nhật Bản; confidence cao trên vài crop không đồng nghĩa accuracy cao tại camera mục tiêu. FPS H100 cũng không phản ánh máy phổ thông và không gồm toàn pipeline.

Đánh giá tiếp theo cần gán thủ công toàn bộ sự kiện trên video test, kể cả xe detector bỏ sót và video không vi phạm. Mỗi nhãn chứa ID cổng và thời điểm cắt theo cùng quy ước điểm neo. Dữ liệu phải chia theo camera/video, không chia ngẫu nhiên frame lân cận. Cần bổ sung ban đêm, mưa,



Hình 4: Chuỗi frame theo dõi và xác nhận xe cắt vạch khi đèn đỏ. Số vàng là chỉ số frame; vạch đỏ là cổng giám sát.

ngược sáng, ùn tắc, xe máy và che khuất. Các mô-đun nên được đo riêng bằng mAP, confusion matrix đèn, IDF1/HOTA và sai số thời điểm cắt trước khi đánh giá đầu-cuối.

9 Kết luận

Bài báo trình bày hệ thống phát hiện phương tiện vượt đèn đỏ có kiến trúc mô-đun, hình học có hướng và bằng chứng truy vết. YOLOv8s đạt cân bằng tốt nhất trên tập con COCO: mAP@0.50:0.95 bằng 0,318, F1 bằng 0,529 và 119,9 FPS với 11,1 triệu tham số. SSD nhanh nhất nhưng kém chính xác; YOLO11x lớn hơn đáng kể mà không cải thiện mAP.

Giá trị chính của hệ thống nằm ở việc hợp nhất nhận diện, theo dõi, trạng thái đèn và quỹ đạo thành quyết định sự kiện có bằng chứng. Tuy nhiên, chỉ số detector chưa đủ để khẳng định độ chính xác toàn pipeline. Hướng tiếp theo là xây dựng dữ liệu camera thực tế, gán nhãn sự kiện độc lập, đánh giá theo camera giữ kín và tối ưu cho điều kiện giao thông Việt Nam.



(a) ID 15, 1,833 giây. (b) ID 25, 2,933 giây.

Hình 5: Ảnh bằng chứng của hai sự kiện. Khung đỏ đánh dấu phương tiện cần kiểm tra; toàn cảnh giữ ngữ cảnh đèn và nút giao.

Tài liệu tham khảo

[1] Joseph Redmon, Santosh Divvala, Ross Girshick và Ali Farhadi. 2016. You Only Look Once: Unified, Real-Time Object Detection. Trong *CVPR*, trang 779–788. https://arxiv.org/abs/1506.02640.

[2] Wei Liu, Dragomir Anguelov, Dumitru Erhan và cộng sự. 2016. SSD: Single Shot MultiBox Detector. Trong *ECCV*, trang 21–37. https://arxiv.org/abs/1512.02325.

[3] Tsung-Yi Lin, Michael Maire, Serge Belongie và cộng sự. 2014. Microsoft COCO: Common Objects in Context. Trong *ECCV*, trang 740–755. https://arxiv.org/abs/1405.0312.

[4] Yifu Zhang, Peize Sun, Yi Jiang và cộng sự. 2022. ByteTrack: Multi-Object Tracking by Associating Every Detection Box. Trong *ECCV*, trang 1–21. https://arxiv.org/abs/2110.06864.

[5] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov và Liang-Chieh Chen. 2018. MobileNetV2: Inverted Residuals and Linear Bottlenecks. Trong *CVPR*, trang 4510–4520. https://arxiv.org/abs/1801.04381.

[6] Alex D. Pon và cộng sự. 2023. Traffic Light Recognition using Convolutional Neural Networks: A Survey. *arXiv:2309.02158*. https://arxiv.org/abs/2309.02158.

[7] Autoware Foundation. 2026. Traffic Light Classifier for Autoware: Model Card. https://huggingface.co/AutowareFoundation/traffic_light_classifier.

Lời cảm ơn

Em xin trân trọng cảm ơn thầy cô giảng dạy môn Deep Learning đã cung cấp nền tảng kiến thức và định hướng để em thực hiện đề tài *Phát hiện phương tiện vượt đèn đỏ*. Em cũng xin cảm ơn cộng đồng mã nguồn mở và các tác giả của những công trình được trích dẫn.

Mục lục

1 Giới thiệu **1**

2 Cơ sở lý thuyết và công trình liên quan **1**
2.1 Nhận diện đối tượng 1
2.2 Theo dõi đa đối tượng 1
2.3 Nhận diện trạng thái đèn 2

3 Phát biểu bài toán **2**

4 Kiến trúc hệ thống **2**
4.1 Pipeline tổng thể 2
4.2 Hiệu chỉnh theo camera 2
4.3 Nhận diện và theo dõi xe 2
4.4 Trạng thái đèn 2
4.5 Cắt vạch và xác nhận 2

5 Cài đặt **3**

6 Thiết kế thực nghiệm **3**
6.1 Dữ liệu và huấn luyện 3
6.2 Chỉ số 3

7 Kết quả và đánh giá **3**
7.1 So sánh định lượng 3
7.2 Chi phí huấn luyện 3
7.3 Đánh giá định tính sự kiện 3

8 Phân tích sai số và hạn chế **4**

9 Kết luận **5**

Tài liệu tham khảo **5**

Tuyên bố sử dụng AI

1. Sử dụng AI hỗ trợ viết báo cáo:
   ☒ Yes ☐ No.
2. Sử dụng AI hỗ trợ sinh các biểu đồ:
   ☒ Yes ☐ No.
3. Sử dụng AI hỗ trợ sinh giao diện:
   ☐ Yes ☒ No.
4. Sử dụng AI hỗ trợ sinh test cases:
   ☒ Yes ☐ No.
5. Sử dụng bao nhiêu % AI: **70%**.